Type Assisted Synthesis of Programs with Algebraic Data Types

Extended Abstract

Jeevana Priya Inala
MIT CSAIL, Cambridge MA
jinala@mit.edu
ACM student member #: 9549241
Category: Undergraduate

Armando Solar-Lezama (advisor)
MIT CSAIL, Cambridge MA
asolar@csail.mit.edu

1. Research Problem

Algebraic data-types(ADT) and pattern matching are foundational concepts in functional programming and are particularly useful in programs that perform symbolic manipulation, such as compilers or program analysis tools. Despite their convenience, however, there are many situations where functions involving ADTs and pattern matching can become large and difficult to write correctly. In this work, we explore how synthesis can help implement non-trivial functions based on pattern matching and algebraic data-types.

One such class of functions is figuring out the desugaring rules from a high level language to a low level language. Here, we expect the correctness specification to be in the form of behavioral constraint defined in terms of the output on an interpreter on the original and desugared languages. Other examples include figuring out optimizations on Abstract Syntax Trees(AST) used in compilers and generating constraints for type inference used in program analysis tools.

The main problem in synthesizing these kinds of benchmarks is that the search space is extremely huge. Hence, it is necessary to have an efficient way of describing the space of possible implementations that the synthesizer should consider and an efficient search strategy.

Our approach to solve this problem is to use a combination of type inference and counterexample-guided inductive synthesis (CEGIS) to greatly reduce the search space.

2. Approach

We build on previous work on constraint-based synthesis from program templates or sketches [5]. The key idea of this work is to leverage the structure provided by the type definitions of an algebraic data-type and use it to synthesize large and potentially complicated routines from very high-level templates.

We first define a new set of synthesis constructs that allow programmers to express the high-level structure of a collection of pattern matching rules. The new synthesis constructs can be understood from the following sketch:

```
dstLang desugar(srcLang src){
  switch(src){
    repeat_case:
      return new ??( 2, {desugar(src .??),  src .??});
}     }
```

The goal of this sketch is to synthesize desugaring rules from a source language to a destination language both of which are defined as ADTs. The sketch also contains test harnesses that constrain the intended behavior (omitted here).

Here, the **switch** statement implements a form of pattern matching. The repeat_case block tells the synthesizer to have a separate case for each variant of the algebraic data-type; any "holes" in the repeat_case can be resolved differently for each case, allowing the body to be specialized for each of the variants.

Within the body of repeat_case, there are two different kinds of holes that are new to this work. The expression ??(2,{...}) in the sketch is a *generalized constructor* which the synthesizer will specialize to a constructor for one of the variants of the ADT—exactly which one will be determined by the synthesizer. The fields of this new value will be initialized either from one of the values passed as an argument in the list, from constants, or with recursively constructed values, where the recursion is bounded by the first argument—2 in the case of the example. The expression src.?? is a *field selector hole*; the synthesizer uses the type of src in each case and the inferred type of the expression to restrict the set of possible fields that ?? can resolve to.

Synthesis in presence of these high level constructs can be transformed to the problem of completing programs with unknown constants [6] by using type information.The challenge in implementing this transformation is that it requires type information to be propagated both top-down and bottom-up. We achieve this by using bi-directional rules that make this flow of information explicit [4].

The rules have the form shown below; the transformation is parameterized by a set of types $T = \{\tau_0 \dots \tau_i\}$, and the result of applying the transformation is a set of expressions $\{e_0 \dots e_k\}$. The transformation guarantees that the type of

Benchmark	Runtime	Memory	Search space
Insertion into binary tree	7.44	204.32	2^{72}
Simple language desugaring	60.06	882.26	2^{110}
Church encodings for booleans	517.92	1747.54	2^{541}
Church encodings for pairs	1662.12	1919.72	2^{183}
Type constraints for λ calculus	10.23	261.88	2^{149}
Optimizations on ASTs	163.09	880.44	2^{162}

Figure 1. Run times(s), Memory consumption(MiB)and search space for various benchmarks

each expression $e : \tau \in \{e_0 \dots e_k\}$ belongs to the set T.

$$\frac{T = \{\tau_0 \dots \tau_i\}}{\Gamma \vdash e \xrightarrow{T} \{e_0 \dots e_k\}}$$

By transforming the expression e under a set of types T, we propagate top down information about the type of expressions required in a given context. For example, the rule of field selector hole $e.??$ finds all the types $T^{'}$ with fields of the desired type and then uses that set of types to transform the base expression e.

$$\frac{\Gamma \vdash e \xrightarrow{T'} \{e_0 \dots e_k\} \quad where\ T^{'} = \{\tau \mid \tau\ has\ a\ field\ l : \tau_i\ and\ \tau_i \in T\ \}}{\Gamma \vdash e.?? \xrightarrow{T} \{e_i.l_j \mid e_i.l_j : \tau\ \ i \in [0, k]\ \ and\ \tau \in T\}}$$

Once the problem is reduced to a problem of synthesizing integer unknowns, we use the standard constraint based approach of synthesis, CEGIS to finish the synthesis process.

3. Results

We evaluated the approach on benchmarks that include synthesizing simple data structure manipulations, desugaring language constructs including desugaring of booleans and pairs to λ calculus., generating type constraints and optimizing ASTs. The results can be seen in Figure 1.

4. Related Work

The most relevant piece of related work is the synthesizer Leon [2] which can synthesize provably correct recursive functions involving algebraic data-types. Even though, it is possible to describe the behavioral specifications like interpreters in Leon, Leon does not scale well on these kinds of benchmarks. This is because the deductive rules that Leon uses to break the complex problem into smaller ones will not significantly reduce the problem when the specification is in the form of a complex program.

Escher [1] and Lazy [3] are other systems focusing on recursive programs but they are pure programming from examples and do not support behavorial constraints.

The solver aided language Rosette [8; 7] can also express all our benchmarks. However, Rosette is a dynamic language and lacks static type information, so in order to get the benefits of the high-level synthesis constructs described here, it would be necessary to re-implement all our machinery as an embedded DSL.

References

[1] A. Albarghouthi, S. Gulwani, and Z. Kincaid. Recursive program synthesis. In *CAV*, pages 934–950, 2013.

[2] E. Kneuss, I. Kuraj, V. Kuncak, and P. Suter. Synthesis modulo recursive functions. In *OOPSLA*, pages 407–426, 2013.

[3] D. Perelman, S. Gulwani, D. Grossman, and P. Provost. Test-driven synthesis. In *PLDI*, page 43, 2014.

[4] B. C. Pierce and D. N. Turner. Local type inference. *ACM Trans. Program. Lang. Syst.*, 22(1):1–44, Jan. 2000.

[5] A. Solar-Lezama. *Program Synthesis By Sketching*. PhD thesis, EECS Dept., UC Berkeley, 2008.

[6] A. Solar-Lezama, L. Tancau, R. Bodik, V. Saraswat, and S. Seshia. Combinatorial sketching for finite programs. In *ASPLOS '06*, San Jose, CA, USA, 2006. ACM Press.

[7] E. Torlak and R. Bodík. Growing solver-aided languages with rosette. In *Onward!*, pages 135–152, 2013.

[8] E. Torlak and R. Bodík. A lightweight symbolic virtual machine for solver-aided host languages. In *PLDI*, page 54, 2014.